

Boosting Encrypted Traffic Classification Using Feature-Enhanced Recurrent Neural Network With Angle Constraint

Gongxun Miao^{ID}, Guohua Wu, Zhen Zhang, Yongjie Tong, and Bing Lu

Abstract—With the surge in various types of network traffic and the widespread application of encryption technology, the classification of encrypted traffic plays an increasingly important role in ensuring network security, enhancing quality of service, and managing network traffic. However, most existing methods often suffer from issues such as excessive reliance on manual feature extraction and expert knowledge, unstable classification performance, and lack of transfer learning capabilities. To address these challenges, this paper proposes a high-performance hybrid encrypted traffic classification framework, FERNN-AC. It directly extracts features from raw traffic and fully explores and utilizes the spatiotemporal information of traffic data by integrating specially designed feature enhancement module and temporal feature extraction module in a reasonable manner. It introduces angle constraints and can be combined with meta-learning, thereby improving classification performance while possessing certain transfer learning capabilities. The experiments are conducted on three datasets, and the results shows that compare with relevant baseline methods, FERNN-AC has excellent and stable classification performance.

Index Terms—Angle constraint, encrypted traffic classification, feature enhancement, few-shot learning, LSTM.

I. INTRODUCTION

AS INTERNET technology advances, the increasingly diverse range of network applications has brought convenience to people's work and daily lives. However, the accompanying surge in various types of network traffic has brought about numerous issues, such as network security threats and a decline in the network quality of service (QoS). Meanwhile, with the rise of awareness about network security and the widespread application of encryption technology, the proportion of encrypted traffic

in the network environment is constantly increasing, gradually becoming an integral component of network traffic [1], [2]. Big data technology offers new possibilities for addressing this challenge. By applying Big Data analytics methods, such as data mining, machine learning, and deep learning, we can extract valuable information from massive amounts of encrypted traffic data, enabling more accurate traffic classification. This is of significant importance for ensuring network security, optimizing network resource utilization, and improving QoS. By mining large-scale historical data and processing real-time streaming data, we can establish more intelligent and dynamic encrypted traffic classification models that adapt to the ever-changing network environment and emerging encryption technologies.

For traffic classification, researchers have proposed various classification methods. For instance, methods based on port lists or payload analysis of data packets are used to identify application types, but these methods are unable to handle encrypted traffic [3]. Statistical-based approaches involve manual feature extraction followed by the utilization of traditional machine learning classifiers for classification [4]. While these methods achieve high accuracy rates, they heavily rely on expert knowledge and manual feature extraction. As the complexity of real network traffic increases, the cost of manual feature extraction becomes prohibitive. Deep learning-based methods can directly use raw traffic as input, offering automatic feature extraction advantages and highly complex nonlinear fitting capabilities. This eliminates the reliance on expert experience during the manual feature extraction process and is more suitable for handling increasingly complex encrypted traffic, making it a hot topic in traffic classification research in recent years [5], [6]. However, these methods still struggle with adapting to uneven sample distributions and distinguishing samples with high similarity across different categories but low similarity within the same category, and lack transfer learning capabilities for new types of samples.

To address the aforementioned issues, this paper proposes an encrypted traffic classification model, FERNN-AC, based on feature-enhanced recurrent neural network framework with angle constraints. FERNN-AC comprises three main modules: feature enhancement, temporal feature extraction, and classification. The feature enhancement and temporal feature extraction modules fully exploit and utilize the spatiotemporal information of traffic data, enabling the model to obtain effective representations. The classification module imposes constraints

Received 8 May 2024; revised 14 September 2024; accepted 16 October 2024. Date of publication 22 October 2024; date of current version 11 July 2025. This work was supported in part by the "Pioneer" and "Leading Goose" Research and Development Program of Zhejiang under Grant 2023C03203, Grant 2023C03180, and Grant 2022C03174. Recommended for acceptance by P. D'Urso. (Corresponding author: Gongxun Miao.)

Gongxun Miao is with the School of Cyberspace, Hangzhou Dianzi University, Hangzhou 310018, China, and also with the Zhongfu Information Company, Ltd., Jinan 250100, China (e-mail: miaogx@hdu.edu.cn).

Guohua Wu is with the School of Cyberspace, Hangzhou Dianzi University, Hangzhou 310018, China, and also with the School of Intelligent Science, Technology, Hangzhou Institute for Advanced Study, Hangzhou 310018, China (e-mail: wugh@hdu.edu.cn).

Zhen Zhang is with the School of Cyberspace, Hangzhou Dianzi University, Hangzhou 310018, China (e-mail: zhangzhen@hdu.edu.cn).

Yongjie Tong and Bing Lu are with the Zhongfu Information Company, Ltd., Jinan 250100, China (e-mail: tongyongjie@zhongfu.net; lubing@zhongfu.net).

Digital Object Identifier 10.1109/TBDATA.2024.3484674

TABLE I
ABBREVIATION EXPLANATIONS

Abbreviation	Full Term	Abbreviation	Full Term
FERNN-AC	Feature-Enhanced Recurrent Neural Network with Angle Constraint	C ³ LR	Class-Cognizant Contrastive Learning
LSTM	Long Short Term Memory	RNN	Recurrent Neural Network
QoS	quality of service	GRU	Gated Recurrent Units
IPsec	Internet Protocol Security	CTU	Czech technical university
TOR	The Onion Router	SE	Squeeze-and-Excitation
ProtoNet	Prototypical Network	CNN	Convolutional Neural Network
MAML	Model Agnostic Meta Learning	HTTP	HyperText Transfer Protocol
L2F	Learned to Forget	L7	Layer 7
VPN	Virtual Private Network	SAE	Stacked Autoencoder

TABLE II
COMPARISON OF RELATED WORKS IN NETWORK TRAFFIC CLASSIFICATION

Method	Spatiotemporal information	Transfer learning	Stability	Computational complexity
RF[33]/XGBoost[34]	○	○	●	●
1D-CNN[10]	●	○	○	●
Deeppacket[14]	●	○	○	○
ET-BERT[16]	●	●	●	○
ST-CausalConvNet*[35]	●	○	●	○
C ³ LR[26]	●	●	○	●
ProtoNet[21]	●	●	●	●
FC-net[23]	●	●	○	○
NAS-based FS-MTC[31]	●	●	○	●
Festic[32]	●	●	○	●
FERNN-AC	●	●	●	●

on the angle in the embedded space and can be combined with few-shot learning methods, thus achieving high-performance classification of traffic and certain transfer learning capabilities. The main contributions of this paper are summarized as follows,

- A hybrid high-performance encrypted traffic classification framework, FERNN-AC, is proposed, which makes full use of the spatiotemporal information of traffic data and imposes angle constraints on the embedding vector, realizing the high performance classification of traffic data.
- A unique feature enhancement module is designed, which integrates one-dimensional pointwise convolution with the Squeeze & Excitation mechanism, effectively utilizing the features of the traffic data and improving the model's capability to the nonlinear representation.
- By aggregating all temporal step information of the recurrent neural network, the global information of encrypted traffic flows are retained, which effectively enhances the model's the representation ability.
- The introduction of L_2 normalization and angular constraints enforces closer proximity between samples of the same type and greater separation between samples of different types, thereby improving classification performance. Additionally, the model can be combined with meta-learning, enabling transfer learning with few-shot case.

We conducted extensive experiments on three real network traffic datasets, and the results demonstrate that FERNN-AC exhibits outstanding classification performance and stability across different datasets.

The rest of this paper is organized as follows: Section II reviews related work in the field of traffic classification; The concepts and problem definitions in the article are described in

Section III; Section IV provides a detailed description to the theoretical framework of FERNN-AC; Section V presents the experimental details, including datasets, experimental settings, and analysis of experimental results; Finally, we conclude this paper in Section VI.

II. RELATED WORK

In this section, we review previous researches on encrypted traffic classification, including statistical-based methods, deep learning-based methods and few-shot learning methods. The professional terms and their corresponding abbreviations mentioned throughout this article are shown in Table I. In Table II, a comparison between the state-of-the-art techniques and the method proposed in this paper was conducted across four characteristics: spatiotemporal information, transfer learning, stability, and computational complexity. The comparison is represented as follows, ● indicates a significant advantage, ● indicates a moderate advantage, and ○ indicates no advantage.

A. Statistical-Based Methods

These methods manually extract a feature set from the traffic based on prior knowledge, then use data containing these feature sets to train traditional machine learning models such as decision trees, random forests, Bayesian classifiers, k-nearest neighbors, etc., to achieve classification. The selected traffic features can be divided into two categories: packet features and network flow features. Packet features include packet size, packet inter-arrival time, packet header length, etc.; network flow features include flow duration, up/downstream byte counts, flow bit rate per second, number of packets, etc. For example, Leroux et al. [7]

extracted features such as packet size, arrival time, burst time, burst size, etc. from encrypted flows, and then used naive Bayes, logistic regression, and random forests to predict traffic types passing through IPsec or TOR tunnels. Obasi et al. [8] proposed an ensemble learning model CARD-B, which stacked multiple technical components using random forests and achieved an overall accuracy of 96% and an AUC value of 98% on the ISCXVPN2016 dataset. Draper-Gil et al. [9] studied the time-related characteristics of flow-based VPN traffic detection and classified encrypted traffic into categories such as web pages and streaming media. The above methods have achieved satisfactory results in encrypted traffic classification tasks, but the extraction of feature sets heavily relies on expert knowledge, and model training typically requires large-scale labeled samples.

B. Deep Learning-Based Methods

These methods typically involve automatic feature learning from ample labeled data, thereby effectively circumventing the manual feature extraction process. For instance, Wang et al. [10] proposed an end-to-end one-dimensional Convolutional Neural Network (CNN) model for encrypted traffic classification, achieving a 17% reduction in false positive rate on the ISCXVPN2016 dataset. Liu et al. [11] introduced an end-to-end Recurrent Neural Network (RNN) traffic classification model called FS-NET. This model took packet length sequences as input and utilized bidirectional Gated Recurrent Units (GRU) to learn traffic feature representations, achieving a recall rate of 99.14% on a real dataset containing 18 applications. Hu et al. [12] combined CNN and Long Short-Term Memory (LSTM) layers to improve the classification performance of unknown encrypted network traffic. Akbari et al. [13] designed features with good generalization and combined stacked LSTM layers with CNN to learn encrypted protocols. Lotfollahi et al. [14] proposed a Deep-Packet encrypted traffic classification model, which combines CNN with stacked autoencoders to classify traffic applications by analyzing packet payloads. Shi et al. [15] proposed a Byte Feature Convolutional Network (BFCN) algorithm based on BERT and CNN for encrypted traffic, which demonstrated significant improvements in recall, precision, and F1-score. The BFCN integrated a transformer-based packet encoder and a CNN module to capture both global traffic characteristics and byte-level local features. Similarly, Lin et al. [16] proposed ET-BERT. The model utilized a transformer encoder to learn feature representations of raw traffic data streams and packets, and it employed unsupervised pre-training and supervised fine-tuning techniques for training and testing. ET-BERT could adapt to multiple downstream tasks with limited labeled samples and achieved state-of-the-art performance across five encrypted traffic classification tasks.

Yang et al. [17] proposed a customized deep learning technique based on gradient backpropagation (GradBP) and applied it to zero-day application detection. As a commercial-grade traffic classification model, GradBP demonstrated excellent performance in both accuracy and computational cost when classifying known types and detecting unknown types on a dataset with over 200 application labels. Song et al. [18] introduced an incremental

and interpretable recurrent neural network for encrypted traffic classification, called I^2 RNN. Unlike traditional classification model training methods, this model aims to learn independent fingerprints for different types of samples, allowing it to handle imbalanced samples and support rapid training of new sample types. I^2 RNN exhibited stable and effective performance on two public datasets while also providing reliable interpretability. However, the model's hyperparameters rely on expert experience and face the risk of fingerprint database obsolescence. Similarly, Cerasuolo et al. [19] proposed an incremental learning model based on fine-tuning techniques, named MEMENTO. Through its memory management module, the model incorporates old sample information when updating training on new samples, significantly improving classification performance for new application types while maintaining stable classification accuracy for old application types.

C. Few-Shot Learning Methods.

Few-shot learning aims to learn general patterns or rules from limited data to achieve effective generalization to unseen samples. Methods based on few-shot learning include data augmentation, generative models, and meta-learning. Among them, meta-learning methods, as an efficient few-shot learning approach, have made significant progress in the field of image classification, such as Relation Net [20], Prototypical Network (ProtoNet) [21], and Model Agnostic Meta Learning (MAML) [22]. With their special architecture, these methods can learn a universal learning strategy by learning shared patterns and structures across tasks, thereby achieving the effect of transfer learning. Xu et al. [23] were the first to apply meta-learning to network intrusion detection. They used raw traffic bytes as input and achieved few-shot traffic classification by training on meta-tasks. Yang et al. [24] proposed an improved traffic classification model, FS-IDS, which enhanced the performance of the model in few-shot classification by integrating raw traffic and traffic statistical features. Shi et al. [25] introduced the "Learned to Forget" (L2F) mechanism into MAML for the first time. L2F dynamically controlled task conflicts during the initialization process of MAML, thereby improved the convergence speed of the model. Shirekar et al. [26] proposed a method called "Class-Cognizant Contrastive Learning" (C^3 LR), which introduced class-level awareness and modified contrastive loss to learn class-level global structures, thereby enhanced the performance of few-shot classification.

Horowicz et al. [27] proposed a few-shot classification method called mini-FlowPics. This method is based on FlowPic [28] and significantly reduces the dimensions of two-dimensional images by aggregating the time and number of packets. While mini-FlowPics reduces training costs and improves classification performance, it is limited to classification tasks within predefined classes and cannot detect unknown samples. Bovenzi et al. [29], [30] explored the use of few-shot learning in mobile applications and IoT traffic classification tasks. They compared various few-shot learning methods, analyzed the sensitivity of the Number of Ways and Number of Shots, the relationship between embedding functions or the size

TABLE III
NOTATIONS

Notations	Description	Notations	Description
N	The number of sample	D'	Dimension of representation vector j
L	Length of sample sequence	l_j	Representation vector of class j
C_1	The number of filters in the first convolutional layer	S_j	Set of data from class j
C_2	The number of filters in the second convolutional layer	n_j	The number of extracted samples for each class
r	Reduction ratio	f_ϕ	Embedding model
h^t	Hidden vector at time t	s	Scaling factor
$\tilde{h}$	Integrated vector of all time steps	θ_j	Angle between sample's embedding vector and l_j
h	Embedding vector	K	The number of types in one batch
w_h	Linear transformation matrix	N_B	The number of data samples in one batch
D	Dimension of h^t	m	Additive angle margin

of the few-shot set and classification performance, and provided directions for optimizing few-shot learning. Zhang et al. [31] proposed a few-shot malicious traffic classification method that combines knowledge transfer and neural architecture search (NAS). This approach not only overcomes the dependency on labeled data and the manual selection of network architecture parameters but also significantly reduces time costs. Zhao et al. [32] introduced a few-shot classification method for IoT traffic, named Festic. This model uses a meta-learning strategy to extract sample representations and employs an integrated similarity comparison module to compute a composite score for the samples to be classified, achieving the final few-shot classification task. Additionally, Festic incorporates a detection module with a threshold, enabling the identification of unknown samples.

III. PROBLEM DEFINITION

This section introduces several concepts and definitions. Table III outlines the notations used in the paper along with their corresponding descriptions.

A. Classification Tasks

The purpose of encrypted traffic classification is to categorize network traffic according to specific granularities, which helps improve the quality of service and precise control of network traffic. Before performing encrypted traffic classification tasks, it is necessary to first determine the granularity of traffic division. There are mainly five granularities for network traffic division: **1)** Encrypted traffic identification, which determines whether the traffic is encrypted or not. **2)** Protocol classification, which categorizes traffic based on encryption protocols (such as SSL, SSH, IPSec protocols, etc.). **3)** Application classification, which classifies traffic according to the associated applications (such as YouKu, WeChat, Skype, QQ, etc.). **4)** Service classification, which categorizes traffic based on the type of service (such as streaming media, instant messaging, etc.). **5)** Web page classification, which identifies web browsing under the HTTPS protocol (such as Baidu search, Taobao, Sohu, etc.). This paper conducts research on encrypted traffic classification using applications or services as the classification granularity.

B. Meta Learning

The form of traffic data can be represented as $\mathcal{D} = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$, where $x_i \in R^D$ and $y_i \in \{1, 2, \dots, M\}$ respectively denote samples and their corresponding labels. The dataset is divided into two parts, the meta-training set $\mathcal{D}_{train}$ and the meta-test set $\mathcal{D}_{test}$. In this paper, the label sets of $\mathcal{D}_{train}$ and the label sets of $\mathcal{D}_{test}$ have no intersection. The meta-learning process consists of two phases, meta-training and meta-testing, each of which comprises multiple episodes. Each episode contains one or more meta-tasks. Meta-task is the basic unit of meta-learning, which can be defined as $\mathcal{T}_i$. In meta-training phase, $\mathcal{T}_i$ randomly selects a subset of labels from $\mathcal{D}_{train}$, with each label having a few samples randomly selected to form the support set and the query set. In meta-testing, meta-task $\mathcal{T}_i$ are generated from $\mathcal{D}_{test}$ in the same manner as during the meta-training phase.

IV. THEORETICAL FRAMEWORK

This section provides a detailed explanation of the FERNN-AC and its design principles. The detailed structure is shown in Fig. 1, which includes three modules, Feature Enhancement, Temporal Feature Extraction, and Classification in a bottom-up manner. First, the Feature Enhancement is used to extract spatial information of the data and obtain multidimensional features. Then, the output from the previous step is reorganized, and multiple features at each time step are sequentially input into LSTM cell according to the temporal order. The hidden layer information of the LSTM is fused and dimensionally reduced to obtain a representation vector of a sample. Finally, the representation vector is projected onto a hypersphere, and the angle constraint method is used for training. The following is a detailed description of each module.

A. Feature Enhancement

We reprocessed each traffic data into a sequence of length L , as described in Section V-B. First, we organize the data into a single-channel one-dimensional grayscale image (format: $1 \times L$). Typically, single-channel data means that each position point has only one feature. This feature is considered shallow and limited in quantity, making it unable to represent the deep features of the samples. Therefore, to enhance the model's representational capacity and its ability to decouple information from the raw data, this paper applies one-dimensional pointwise

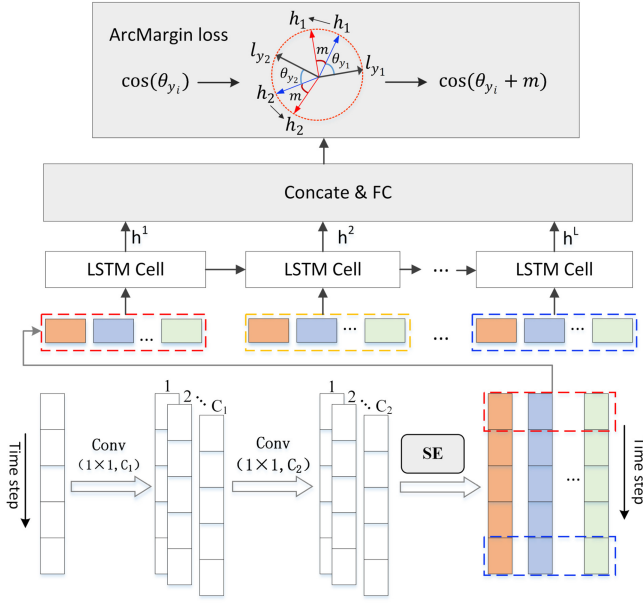


Fig. 1. The overview of FERNN-AC framework.

convolution and performs channel number transformation to achieve this objective. The convolution involves using a 1×1 convolution kernel, and its convolution operation is represented by (1).

$$p_i = f_1 \left(\sum_c w_c x_{i,c} \right) \quad (1)$$

Here w_c represents the convolution kernel parameters, $x_{i,c}$ denotes the input data, i is the position index, and c is the channel index. f_1 is a non-linear function, and ReLU is used here. The stride is set to 1 during the process. We set up two layers of the aforementioned convolutional layers. The first layer employs C_1 convolutional kernels, while the second layer uses C_2 convolutional kernels. The final output is the convolution $p \in R^{C_2 \times L}$.

In general, if a data point consists of multiple features, different features may contribute differently to the classification task. Inspired by the work of [36], this paper introduces the Squeeze-and-Excitation mechanism (SE). Through simple pooling and fully connected operations, the SE mechanism adaptively allocates importance weights to each channel, enhancing the model's ability to utilize different channel features.

The specific structure of the SE module is illustrated in Fig. 2, which consists of two steps: squeeze and excitation. In the squeeze step, global average pooling is performed on each channel, which squeeze the feature maps of each channel into a single value. In the excitation step, two fully connected layers, a dimension reduction layer and a dimension expansion layer (as shown in the second and third boxes of the right branch in Fig. 2) are utilized to learn the weights for each channel. These weights represent the importance of each channel's feature maps. Finally, each learned weight is multiplied element-wise with the

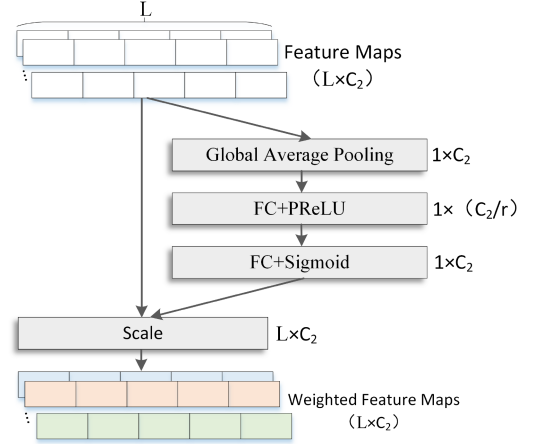


Fig. 2. Structure of SE module.

corresponding feature map in the original feature maps, resulting in the weighted feature maps.

In Fig. 2, r represents the reduction ratio, which is usually an integer greater than 1 that can divide C_2 . PReLU refers to a parameterized nonlinear operation similar to leakyReLU, but with different learnable slope parameters for the negative part of the activation, which vary across different neurons (for detailed explanation, refer to [37]). The “Scale” operation multiplies the feature map of each channel with the corresponding weights obtained from the SE module.

B. Temporal Feature Extraction

Despite the fact that traffic data can be treated as a image and convolutional operations can be applied to extract features, its essence lies in being a sequence, thus temporal characteristics should be taken into consideration. In this paper, LSTM is employed as the temporal feature extraction module, where the data processed by the feature enhancement module are sequentially fed into the LSTM, as shown in Fig. 1. The obtained hidden vectors at each time step are as follows:

$$h^t = LSTM(p^t, h^{t-1}) \quad (2)$$

It is worth noting that when we input data into the LSTM, we do not directly feed the outputs from feature enhancement module in different channels. Instead, the data is first organized into feature vectors at different time steps. Since the features at the same time step contain the characteristics of different channels, the feature vector p^t at any time step t is arranged as $[p_1^t, p_2^t, \dots, p_{C_2}^t]$, where C_2 represents the number of channels output by the convolutional operation. Then, the feature vectors at each time step are sequentially input into the corresponding time steps of the LSTM in chronological order.

After obtaining the hidden vectors h^t at each time step, they are concatenated to form a representation vector $\tilde{h}$ that contains global information, where $\tilde{h} = h^1 \oplus h^2 \dots \oplus h^L$ and $h^t \in R^D$. Then, a fully connected layer is applied to fuse and reduce the dimensionality of $\tilde{h}$.

$$h = W_{\tilde{h}} \cdot \tilde{h} \quad (3)$$

Where $\tilde{h} \in R^{L \cdot D}$, and $W_{\tilde{h}} \in R^{D' \times (L \cdot D)}$. Here, D and D' represent the dimensions of the hidden vector's and the embedding vector's dimensions respectively.

C. Classification

After the processing in Sections IV-A and IV-B, each traffic data is transformed into an embedding vector in a latent space. Traditional classification methods often involve using a simple fully connected layer and softmax loss for classification, where the classification matrix W and bias term b are adjusted through training data. However, for certain traffic data, there are great differences among samples from same type, and great similarities among different types of samples. This results in suboptimal classification performance for some classes. Moreover, with the increasing diversity of applications and advancements in encryption techniques, newly generated traffic may exhibits certain differences from the past, and some types of traffic may be insufficient in quantity, leading to insufficient generalization of models trained solely on historical data.

Inspired by articles [38], [39], We alleviate the above issues by introducing angle constraints into the embedded space. Without considering the bias term b , each row of the classification matrix W can be viewed as a vector representing the class-specific features (referred to as "class representation vectors" and denoted as l_j , where $W_j = l_j^T$). The process of classifying a data point can be seen as taking the dot product between its embedding vector and the class representation vectors, and selecting the class with the highest dot product as the predicted class for this data point. To enhance the stability of the model's classification, we eliminate the influence of feature scale differences [40], [41] by applying L_2 normalization to both the embedding vectors and the class representation vectors. Specifically, $W_j h_i = \|W_j\| \|h_i\| \cos \theta_j$, and applying L_2 normalization to W_j and h_i allows $\|W_j\| = \|h_i\| = 1$. We then scale it with a factor s , so that all embedding vectors and class representation vectors lie on the surface of a sphere with a radius of s .

The classification matrix W can be randomly initialized and learned through training, similar to traditional methods. However, it can also be made to rely not only on the training data but also on external memory, which can partially address the issues of limited generalization and lack of transfer learning ability caused by the scarcity of certain types of data in real-world scenarios. Specifically, drawing from the idea of meta-learning [20], [21], we generate the class representation vectors in real-time based on the data included in the current task. In each batch, a certain number of data samples are selected for each class to form the class representation vectors, following the process outlined below:

$$l_j = \sum_{x_i \in S_j} f_\phi(x_i) \quad (4)$$

Here, l_j represents the class representation vector for the j -th class, S_j denotes the set of data belonging to class j , and the number of samples extracted from S_j for each class is denoted as n_j . f_ϕ refers to the embedding model (i.e., the process described in Sections IV-A and IV-B), where the embedding vector

TABLE IV
SUMMARY OF THE DATASET USED FOR EXPERIMENT

DataSets	Number of sessions		Number of categories
	train	test	
ISCXVPN2016	42055	5253	12
USTC-TFC2016	72352	9070	20
App60	229409	28676	60

$h_i = f_\phi(x_i)$. In the experiments conducted in Section V, we will compare and evaluate the performance of the classification matrix generated using these two different approaches.

According to the research [42], the paper introduces an additive angular margin penalty in the cosine space to impose certain constraints on the distribution of data's representation vectors on the hypersphere, which enhances intra-class compactness and inter-class discrepancy simultaneously (this method is called ArcMargin). The final model's loss function can be written as follows,

$$\mathcal{L} = -\frac{1}{N_B} \sum_{i=1}^{N_B} \log \left(\frac{e^{s \cos(\theta_{y_i} + m)}}{e^{s \cos(\theta_{y_i} + m)} + \sum_{j=1, j \neq y_i}^K e^{s \cos \theta_j}} \right) \quad (5)$$

Where θ_{y_i} is the angle between the sample and the class representation vector it belongs to, N_B is the number of samples in a batch during the training process, K is the number of classes, s is the scaling factor, and m is the additive angle margin.

D. Complexity Analysis

The computational complexity of two-layer convolution and SE module are $O(L \cdot C_1 \cdot C_2)$ and $O(C_2^2/r)$, respectively. LSTM and fully connected layer need complexity $O(L \cdot (C_2 \cdot D + D^2))$ and $O(L \cdot D \cdot D')$, respectively. The classification layer costs $O(D' \cdot M)$. D and C_2 are much smaller than D' . C_2/r is less than C_1 . M significantly smaller than $L \cdot D$. Thus, the overall time complexity is $O(L \cdot C_1 \cdot C_2 + L \cdot D \cdot D')$.

V. EXPERIMENTS AND ANALYSIS

A. Datasets

In order to validate the performance of the proposed method, experiments were conducted on three datasets: ISCXVPN2016, USTC-TFC2016, and App60. Table IV shows the number of samples and types after preprocessing for the three datasets. Descriptions of the datasets are as follows,

- **ISCXVPN2016:** It consists of encrypted traffic data comprising both Virtual Private Network (VPN) and non-VPN traffic [9]. This dataset comprises pcap files corresponding to 17 types of applications, including Skype, Email, Facebook, Gmail, and others. Among these, the SCP, Tor, and Gmail applications only contain non-VPN packets, while the remaining 14 application types contain both VPN and non-VPN packets. The application types are further categorized into 12 classes based on the specific activities involved in the packets of each application, such as voice calls, chats, etc.

- *USTC-TFC2016*: It consists of encrypted traffic containing both malicious software and benign applications [43]. This dataset comprises 20 categories of application flows, with 10 categories for benign applications and 10 for malicious ones. The malicious software traffic was collected by CTU researchers from real network environments, while the normal software traffic was collected using traffic simulation device. The dataset size is 3.71 GB.
- *App60*: The dataset is collected from 60 mobile applications (such as YouKu, QQ, PPTV, Douyin, Gmail, etc.) using the Monkey tool to trigger specified applications on Android devices from Google Play [44]. The dataset has a total size of 32.3 GB and consists of raw traffic and flow files.

B. Data Preprocessing

The characteristic information of network traffic data is typically reflected at the application layer (L7) of the OSI model. For instance, the SMTP protocol represents email traffic, and the HTTP protocol represents browser traffic. According to the research in [45], using information from application layer can effectively avoid biased input and improve the generalization ability of the model. Therefore, this paper utilizes “Session+L7” of raw traffic as the representation for each traffic sample, where “Session” denotes the bidirectional flow of traffic, and “L7” denotes application layer of traffic.

The method described in article [43] was adopted for data preprocessing in this study. First, the raw traffic was divided into sessions, with each session representing a sample. Second, the first L bytes were extracted from each session, and if the total number of bytes in a session was less than L , padding with zeros was performed. Therefore, each sample was organized into a sequence of length L . Finally, these hexadecimal bytes were converted into decimal values, ranging from 0 to 255.

C. Evaluation Metrics

This paper employs precision (P), recall (R), accuracy (Acc), and F1 score (F1) to evaluate the performance of the model. The evaluation metrics are defined as (6)–(9),

$$P = \frac{1}{M_{test}} \sum_i P_i \quad (6)$$

$$R = \frac{1}{M_{test}} \sum_i R_i \quad (7)$$

$$F1 = \frac{1}{M_{test}} \sum_i F1_i \quad (8)$$

$$Acc = \frac{1}{N_{test}} \sum_{i=1}^{M_{test}} TP_i \quad (9)$$

where, $P_i = \frac{TP_i}{TP_i + FP_i}$, $R_i = \frac{TP_i}{TP_i + FN_i}$, $F1_i = \frac{2P_i R_i}{P_i + R_i}$ represent the precision, recall and F1 score of the i -th class sample respectively. TP_i represents the number of i -th class samples identified as the i -th class, FP_i represents the number of non- i -th class samples identified as the i -th class, and FN_i represents the number of i -th class samples identified as non- i -th class.

TABLE V
EXPERIMENTAL PARAMETERS

Parameter	Value	Parameter	Value
L	128	r	4
C_1/C_2	64/32	D	64
n_j	10	D'	512
s	30	m	0.8/1.0/0.4

M_{test} and N_{test} are respectively the number of categories and the number of samples in the test.

D. Experimental Setup

During training, we applied additive angular margin penalty to the angle between the samples and their corresponding class representation vectors. During testing, we directly performed dot product between the normalized representation vectors of the samples and the class representation vectors to obtain the classification results. This paper conducted experiments on conventional classification (Experiment 1), i.e., training on a large dataset containing numerous samples and then testing on a test set (with the same class set as the training set). As class representation vectors can be generated from real-time data, this paper combined the model with a meta-learning framework and conducted experiments on transfer learning under few-shot scenarios (Experiment 2).

- *Experiment 1*: In this type of experiment, we set up training and testing sets similar to conventional classification methods. The samples in the training and testing sets are different, but their set of classes is exactly the same. There is no scenario involving transfer learning. In this experiment, we configured the model in two ways: one where class representation vectors are randomly generated by the system and trainable, denoted as FERNN-AC(GT); and another where class representation vectors are sourced from external memory, i.e., generated from a few number of real-time data or standardized data, denoted as FERNN-AC(EM). We selected R , P , F_1 and Acc as evaluation metrics for this experiment.
- *Experiment 2*: In this type of experiment, the set of classes in the training differ from those in the testing. Additionally, due to the limited number of samples available for classifying new types during testing, this type of classification can be considered as transfer learning based on few-shot sizes. For ISCXVPN2016, USTC-TFC2016, and App60, the numbers of classes in the training and testing sets were chosen as (7, 5), (12, 8), and (45, 15), respectively, with no intersection between them. They respectively contain combinations of C_{12}^7 , C_{20}^{12} and C_{60}^{45} . For each dataset, we randomly selected 10 combinations and conducted 5-shot 5-way tests, averaging the Acc and F1 scores over all combinations of the testing sets as the evaluation metrics for this experiment.
- *Parameter Settings*: The detailed explanation of the parameters is given in Table V, where the three values of m respectively represent the settings on ISCXVPN2016, USTC-TFC2016 and App60.

TABLE VI
COMPARISON OF CLASSIFICATION RESULTS BETWEEN THE PROPOSED METHOD AND ALL BASELINE METHODS

Method	ISCXVPN2016				USTC-TFC2016				App60			
	Acc	P	R	F1	Acc	P	R	F1	Acc	P	R	F1
RF ^[33]	0.8418	0.7975	0.7868	0.7917	0.8853	0.8879	0.8873	0.8872	0.5327	0.5134	0.5029	0.5054
XGBoost ^[34]	0.8588	0.8113	0.8023	0.8062	0.9037	0.9139	0.9052	0.9062	0.7199	0.7291	0.6911	0.7017
1D-CNN ^[10]	0.8342	0.7924	0.7769	0.7694	0.9003	0.9201	0.8982	0.8965	0.7079	0.6627	0.6255	0.5934
DeepPacket ^[14]	0.5219	0.4699	0.4997	0.4108	0.7018	0.7048	0.6631	0.6313	0.3841	0.4325	0.3683	0.3568
ET-BERT ^[16]	0.9633	0.9447	0.9442	0.9444	0.9622	0.9601	0.9619	0.9601	0.9212	0.9249	0.9210	0.9215
ST-CausalConvNet ^{*[35]}	0.9556	0.9350	0.9362	0.9350	0.9525	0.9523	0.9518	0.9514	0.8612	0.8622	0.8551	0.8569
FERNN-AC(EM)	0.9640	0.9461	0.9489	0.9472	0.9100	0.9114	0.9105	0.9099	0.8934	0.8951	0.8946	0.8936
FERNN-AC(GT)	0.9617	0.9439	0.9426	0.9431	0.9622	0.9631	0.9623	0.9615	0.9218	0.9272	0.9181	0.9204

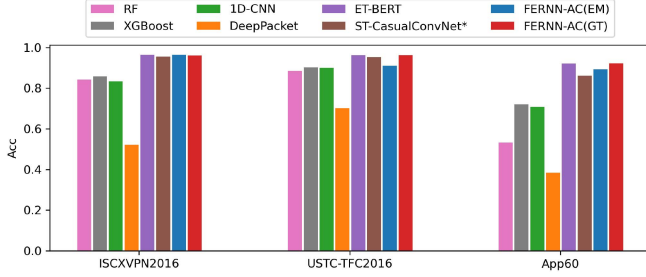


Fig. 3. Acc on three datasets.

E. Results Analysis.

1) *Analysis of Experiment 1:* In this section, we compare the proposed method with relevant baseline methods on three datasets and conduct analysis. The results are shown in Table VI.

- *RF/XGBoost:* These two methods, based on statistical approaches, show moderate performance on the ISCXVPN2016 and USTC-TFC2016. But on the App60, RF and XGBoost performs poorly. Overall, XGBoost outperforms RF. Their inputs consist of 80-dimensional flow features extracted by CICFlowMeter, with the selection of these features requiring domain expertise.
- *1D-CNN:* This method performs moderately on the ISCXVPN2016 and USTC-TFC2016 but poorly on the App60. It adopts a one-dimensional LeNet structure, which despite considering the temporal aspects of traffic data and potentially enhancing classification accuracy, falls short in capturing comprehensive sequence features due to its simple convolutional stacking. With datasets containing a larger variety of types, it struggles to obtain effective representations, resulting in significant performance discrepancies across different datasets.
- *DeepPacket:* This method yields the poorest performance across the three datasets, likely due to the characteristics of the data. As only the application layer information of the packets was retained and the first 1500 bytes of each packet were extracted, a significant portion of the sample sequences contained zero values. These sequences lack sufficient informative content, resulting in class representations that are not adequately representative.
- *ET-BERT:* This model demonstrates excellent and stable performance across all metrics on the three datasets. Especially on App60, it achieve best results in recall and

accuracy. The main reason is that it uses BERT architecture, which undergoes extensive pre-training on massive amounts of traffic data containing diverse types, enabling it to acquire more effective representations of traffic data. Subsequently, fine-tuning is performed on downstream tasks on various datasets, making it more adaptable to each classification task.

- *ST-CausalConvNet*:* This model achieves generally better results across all three datasets. It is based on a causal convolution structure, which possesses the characteristic of obtaining a larger receptive field with the same parameters, thus increasing its ability to perceive sequence characteristics. Additionally, to fully utilize global sequence information, we modified the original structure of the model to fuse and utilize all information from the last convolutional layer, rather than just that from the last neuron (The label “*” represents this modification). These two aspects may be the main reasons for its outstanding performance.
- *FERNN-AC(EM)/(GT):* This model achieves the best results across all metrics on the ISCXVPN2016 and USTC-TFC2016. It is on par with ET-BERT on the App60. The main reason for its performance lies in its design tailored to the characteristics of traffic data. First, It employs a feature enhancement module that maximizes the utilization of data features. And then it utilizes LSTM to extract and fuse the temporal features of the data, obtaining a globally perceptive embedding vector. Finally, by employing angular margin penalty, the model enhances intra-class compactness and inter-class discrepancy simultaneously. In the comparison between versions GT and EM of the model, on the ISCXVPN2016, FERNN-AC(EM) performs better, while on the USTC-TFC2016 and App60, FERNN-AC(GT) achieves better results. This indicates that the FERNN-AC(EM) excels at handling data with fewer types, while the FERNN-AC(GT) method has an advantage on datasets with a greater variety of types.

We compare Acc on three datasets, as shown in Fig. 3. On the ISCXVPN2016, compared with all baselines, FERNN-AC(EM) achieves the best results, surpassing RF by 14.52%, XGBoost by 12.25%, ET-BERT by 0.07% and ST-CausalConvNet* by 0.88%, respectively. On the USTC-TFC2016, FERNN-AC(GT) is on par with ET-BERT, surpassing XGBoost by 6.47%, 1D-CNN by 6.88%, and ST-CausalConvNet* by 1.02%. On App60, FERNN-AC(GT) shows significant advantages, surpassing XGBoost by 28.05%, 1D-CNN by 30.22%, ET-BERT by

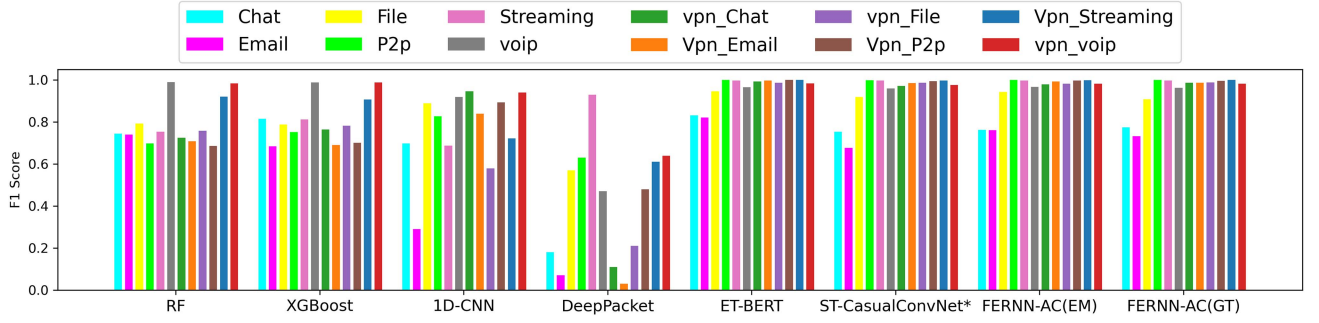


Fig. 4. F1 scores of FERNN-AC and baseline methods across different types on ISCXVPN2016.

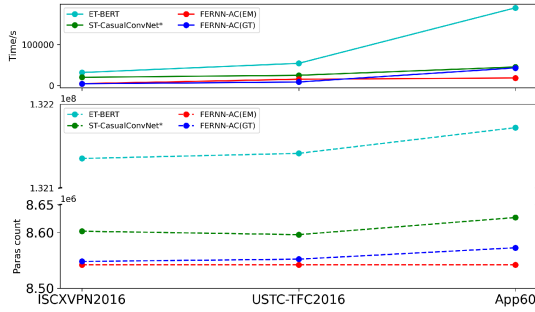


Fig. 5. Comparison of training time and the number of trainable parameter between FERNN-AC and related baseline methods on three datasets.

0.07% and ST-CasualConvNet* by 7.04%, respectively. Overall, the proposed model demonstrates consistently outstanding classification performance.

In addition, to analyze the specific classification details of FERNN-AC and baselines for different types of traffic, we take the example of ISCXVPN2016 (includes 12 classes) and display the F1 score of each class for all methods, as shown in Fig. 4, where different colors represent different types. It can be observed that although RF, XGBoost, 1D-CNN, and DeepPacket show good discriminative capabilities in few classes, their overall performance varies greatly across different categories. ET-BERT, ST-CasualConvNet* and FERNN-AC perform exceptionally well in all categories except for chat and email, possibly due to the lack of distinctiveness in these two categories caused by the traffic's L7 layer features. Overall, FERNN-AC (EM) achieves the best results, surpassing RF, XGBoost, ET-BERT, and ST-CasualConvNet* by 19.64%, 17.49%, 0.30%, and 1.30%, respectively.

Based on the above analysis, it can be observed that the performance of ET-Bert, ST-CasualConvNet*, and FERNN-AC far exceeds that of RF, XGBoost, 1D-CNN and DeepPacket. Therefore, we compare and analyze the training time and the number of trainable parameters of ET-Bert, ST-CasualConvNet*, and FERNN-AC. As shown in Fig. 5, the upper and lower graphs represent the training time and the number of trainable parameters of these methods on three datasets. It can be seen that in terms of training time, FERNN-AC has a significant advantage over

TABLE VII
THE ACC AND F1 SCORES OF THE ABLATION EXPERIMENTS ON THE THREE DATASETS

Method	ISCXVPN2016		USTC-TFC2016		App60	
	Acc(%)	F1(%)	Acc(%)	F1(%)	Acc(%)	F1(%)
Without AC	94.69	92.54	94.75	94.65	88.04	87.68
Without SE	95.77	93.80	96.05	95.98	91.02	90.90
FERNN-AC(GT)	96.17	94.31	96.22	96.15	92.18	92.04
Without AC	92.56	89.20	89.12	88.57	49.45	48.07
Without SE	94.23	94.37	90.73	90.81	89.05	88.97
FERNN-AC(EM)	96.40	94.61	91.00	90.99	89.34	89.36

ET-Bert and ST-CasualConvNet*. On ISCXVPN2016, USTC-TFC2016 and App60, FERNN-AC (GT) is 13.59%, 16.03% and 22.73% of ET-Bert's training time, respectively. In terms of the number of trainable parameters, FERNN-AC (GT/EM) also has the lowest count. Compared to ST-CasualConvNet*, FERNN-AC has higher overall classification performance and fewer parameters. ET-Bert has classification performance similar to FERNN-AC, but the number of trainable parameters far exceeds that of FERNN-AC, being over 15 times more than FERNN-AC on three datasets.

In addition, we conducted ablation experiments by comparing two versions of the FERNN-AC method with their respective counterparts lacking the AC enhancement (without AC) and lacking the SE enhancement (without SE) from Table VII. On the ISCXVPN2016 and USTC-TFC2016, FERNN-AC(GT) demonstrates improvements of over 1.6% in Acc compared to the scenario without AC, while showing only marginal gains in comparison to the scenario without SE. On the App60, FERNN-AC(GT) achieves an approximately 5% increase in both Acc and F1 scores over the scenario without AC, and around 1.3% improvement compared to the scenario without SE.

Compared to the scenario without AC, FERNN-AC(EM) achieves notable gains in both Acc and F1 scores across all datasets, with over an 80% increase on the App60. In comparison to the scenario without SE, the improvements were less pronounced, with the highest increase in Acc at 2.3% and in F1 at 0.4%. Overall, these improvements contribute positively to enhancing the model's performance.

2) *Analysis of Experiment 2:* In this section, we analyze the performance of the model on few-shot classification across three datasets, as shown in Table VIII.

TABLE VIII
THE FEW-SHOT CLASSIFICATION RESULTS OF DIFFERENT METHODS ON THE THREE DATASETS

Method	ISCXVPN2016		USTC-TFC2016		App60	
	Acc	F1	Acc	F1	Acc	F1
C ³ LR[26]	0.6334	0.6068	0.8038	0.7976	0.5991	0.591
ProtoNet[21]	0.8062	0.7964	0.8103	0.8047	0.7598	0.7506
FC-net[23]	-	-	0.7884	0.7704	0.6020	0.5891
NAS-based FS-MTC[31]	0.6215	0.6047	0.7825	0.7794	0.6628	0.6413
Festic[32]	0.4549	0.4386	0.5534	0.5393	0.4112	0.3902
FERNN-AC(FS)	0.8096	0.7986	0.8258	0.8123	0.6746	0.6585

- *C³LR*: It performs moderately on ISCXVPN2016 and USTC-TFC2016, and poorly on App60. One possible explanation for this is that it ignores the sequential features of the data and its pretraining approach is not suitable for traffic data. As a result, the model fails to acquire effective representations on traffic data.
- *ProtoNet*: This method demonstrates consistently excellent performance on all three datasets, especially achieving the best results on the App60 dataset. The main reasons for this may be that data augmentation processing is implemented before inputting the data into the model, and the model represents class's features using prototype vectors and is trained using a loss based on euclidean distance. This allows the model to rely more on the characteristics of the real-time data itself rather than the model parameters obtained on the training data, thus enabling the model to be more suitable for datasets that include a greater variety of categories.
- *FC-Net*: Due to the inability to preprocess the ISCXVPN2016 dataset, this method was only experimented with on the USTC-TFC2016 and App60 datasets. It doesn't demonstrate superior performance, primarily because it relies on a data format that requires truncating a portion of bytes for each packet. For some datasets, this approach may overlook important features for classification, making it difficult for the model to obtain effective representations.
- *NAS-based FS-MTC*: This method shows moderate performance across all datasets. The main issue is that it uses a convolutional neural network structure, which only captures the spatial structure of traffic data and overlooks important temporal features.
- *Festic*: This method performs poorly across all three datasets, which may primarily be attributed to its corresponding preprocessing approach. The application layer data in the first N packets is either too short or nonexistent, resulting in many zero values in the extracted sequences, making it difficult to learn effective class representations. This also indicates that the method is highly affected by the data.
- *FERNN-AC(FS)*: The proposed model achieves the best results on the ISCXVPN2016 and USTC-TFC2016 datasets, and performs better on App60. The main reason could be that although the data, dealt with feature enhancement and processed by recurrent neural networks, largely preserves the spatial and temporal characteristics of the data, this processing relies heavily on network parameters, which

are highly dependent on training data. As a result, the model may lack generalization ability when faced with new data from a large number of categories. In addition, the model did not implement data augmentation processing like ProtoNet, which could also be a reason for this.

Overall, FERNN-AC(FS) shows better performance compared to the baseline methods. It outperforms the second-best model by 0.42% on ISCXVPN2016 and 1.91% on USTC-TFC2016 in terms of Acc. This indicates that the proposed method not only performs significantly well in conventional classification tasks but also exhibits some transfer learning capabilities.

VI. CONCLUSION

The rapid and accurate classification of encrypted traffic is of great importance for ensuring network security and enhancing network service quality. Based on the nature of encrypted traffic data, this paper proposes a high-performance encrypted traffic classification framework called FERNN-AC. First, the framework employs the feature enhancement module to extract feature of traffic data. The module uses the pointwise convolutional layers to increase the model's non-linear representation capability and obtain multidimensional features, and leverage SE modules to adaptively learn an importance weight for each channel, enhancing the model's ability to utilize different features. Next, the framework employs LSTM to capture the temporal characteristics of the data and performs feature fusion and dimensionality reduction, obtaining an effective embedding vector. Finally, by using additive angular loss, the framework makes the data in the embedding space more compact within the same class and further apart between different classes, thereby improving the classification performance. In this process, the framework can also combine with external memory, so that it can adapt to uneven distribution or conduct transfer learning based on few-shot case. In this paper, experiments are performed on three datasets, and the results show that compared with the baseline methods, FERNN-AC has excellent and stable classification performance.

However, the method proposed in this paper also has some limitations, 1) The model's adaptability and scalability are weak; 2) The model lacks interpretability. The nature of network traffic typically changes rapidly over time, requiring models to have stronger adaptability. Deep learning-based traffic classification models often exhibit a "black box" nature, leading to a lack

of a certain degree of trustworthiness in the classification results. Therefore, in future work, we will focus primarily on enhancing the model's adaptability and interpretability. With the continuous development of artificial intelligence and computing power, we will continue to explore AI-powered network traffic classification models to achieve adaptive enhanced training and dynamic analysis of the factors influencing classification performance.

ACKNOWLEDGMENT

Support for this project is provided by Guohua Wu. The authors thank the anonymous engineers for their support with the software.

REFERENCES

- [1] Z. ThreatLabz, "Zscaler threatlabz 2023 state of encrypted attacks report," Zscaler, Tech. Rep., 2023. [Online]. Available: <https://resources.wisdominterface.com/wp-content/uploads/2024/03/threatlabz-2023-state-of-encrypted-attacks-report.pdf>
- [2] Google, "Iitps encryption on the web – google transparency report," 2024. [Online]. Available: <https://transparencyreport.google.com/https/overview>
- [3] A. Dainotti, A. Pescapé, and K. C. Claffy, "Issues and future directions in traffic classification," *IEEE Netw.*, vol. 26, no. 1, pp. 35–40, Jan./Feb. 2012.
- [4] S. Bagui, X. Fang, E. Kalaimannan, S. C. Bagui, and J. Sheehan, "Comparison of machine-learning algorithms for classification of VPN network traffic flow using time-related features," *J. Cyber Secur. Technol.*, vol. 1, no. 2, pp. 108–126, 2017.
- [5] H. Yao, C. Liu, P. Zhang, S. Wu, C. Jiang, and S. Yu, "Identification of encrypted traffic through attention mechanism based long short term memory," *IEEE Trans. Big Data*, vol. 8, no. 1, pp. 241–252, Feb. 2022.
- [6] B. Wang, J. Zhang, Z. Zhang, L. Pan, Y. Xiang, and D. Xia, "Noise-resistant statistical traffic classification," *IEEE Trans. Big Data*, vol. 5, no. 4, pp. 454–466, Dec. 2019.
- [7] S. Leroux, S. Bohez, P.-J. Maenhaut, N. Meheus, P. Simoens, and B. Dhoedt, "Fingerprinting encrypted network traffic types using machine learning," in *Proc. IEEE/IFIP Netw. Operations Manage. Symp.*, 2018, pp. 1–5.
- [8] T. Obasi and M. O. Shafiq, "CARD-B: A stacked ensemble learning technique for classification of encrypted network traffic," *Comput. Commun.*, vol. 190, pp. 110–125, 2022.
- [9] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related," in *Proc. 2nd Int. Conf. Inf. Syst. Secur. Privacy*, 2016, pp. 407–414.
- [10] W. Wang, M. Zhu, J. Wang, X. Zeng, and Z. Yang, "End-to-end encrypted traffic classification with one-dimensional convolution neural networks," in *Proc. IEEE Int. Conf. Intell. Secur. Informat.*, 2017, pp. 43–48.
- [11] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-Net: A flow sequence network for encrypted traffic classification," in *Proc. IEEE Conf. Comput. Commun.*, 2019, pp. 1171–1179.
- [12] X. Hu, C. Gu, and F. Wei, "CLD-net: A network combining CNN and LSTM for internet encrypted traffic classification," *Secur. Commun. Netw.*, vol. 2021, pp. 1–15, 2021.
- [13] I. Akbari et al., "Traffic classification in an increasingly encrypted web," *Commun. ACM*, vol. 65, no. 10, pp. 75–83, 2022.
- [14] M. Lotfollahi, M. Jafari Siavoshani, R. Shirali Hossein Zade, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft Comput.*, vol. 24, no. 3, pp. 1999–2012, 2020.
- [15] Z. Shi, N. Luktarhan, Y. Song, and G. Tian, "BFCN: A novel classification method of encrypted traffic based on BERT and CNN," *Electronics*, vol. 12, no. 3, 2023, Art. no. 516.
- [16] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, 2022, pp. 633–642.
- [17] L. Yang, A. Finamore, F. Jun, and D. Rossi, "Deep learning and zero-day traffic classification: Lessons learned from a commercial-grade dataset," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 4, pp. 4103–4118, Dec. 2021.
- [18] Z. Song et al., "T²RNN: An incremental and interpretable recurrent neural network for encrypted traffic classification," *IEEE Trans. Dependable Secure Comput.*, to be published, doi: [10.1109/TDSC.2023.3245411](https://doi.org/10.1109/TDSC.2023.3245411).
- [19] F. Cerasuolo et al., "MEMENTO: A novel approach for class incremental learning of encrypted traffic," *Comput. Netw.*, vol. 245, 2024, Art. no. 110374.
- [20] F. Sung, Y. Yang, L. Zhang, T. Xiang, P. H. Torr, and T. M. Hospedales, "Learning to compare: Relation network for few-shot learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 1199–1208.
- [21] J. Snell, K. Swersky, and R. Zemel, "Prototypical networks for few-shot learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 4080–4090.
- [22] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2017, pp. 1126–1135.
- [23] C. Xu, J. Shen, and X. Du, "A method of few-shot network intrusion detection based on meta-learning framework," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 3540–3552, 2020.
- [24] J. Yang, H. Li, S. Shao, F. Zou, and Y. Wu, "FS-IDS: A framework for intrusion detection based on few-shot learning," *Comput. Secur.*, vol. 122, 2022, Art. no. 102899.
- [25] Z. Shi, M. Xing, J. Zhang, and B. H. Wu, "Few-shot network intrusion detection based on model-agnostic meta-learning with L2F method," in *Proc. IEEE Wireless Commun. Netw. Conf.*, 2023, pp. 1–6.
- [26] O. K. Shirekar and H. Jamali-Rad, "Self-supervised class-cognizant few-shot classification," in *Proc. IEEE Int. Conf. Image Process.*, 2022, pp. 976–980.
- [27] E. Horowicz, T. Shapira, and Y. Shavitt, "Self-supervised traffic classification: Flow embedding and few-shot solutions," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 3, pp. 3054–3067, Jun. 2024.
- [28] T. Shapira and Y. Shavitt, "FlowPic: A generic representation for encrypted traffic classification and applications identification," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 2, pp. 1218–1232, Jun. 2021.
- [29] G. Bovenzi, D. Di Monda, A. Montieri, V. Persico, and A. Pescapé, "Few shot learning approaches for classifying rare mobile-app encrypted traffic samples," in *Proc. IEEE Conf. Comput. Commun. Workshops*, 2023, pp. 1–6.
- [30] G. Bovenzi, D. Di Monda, A. Montieri, V. Persico, and A. Pescapé, "Classifying attack traffic in IoT environments via few-shot learning," *J. Inf. Secur. Appl.*, vol. 83, 2024, Art. no. 103762.
- [31] X. Zhang et al., "Enhanced few-shot malware traffic classification via integrating knowledge transfer with neural architecture search," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 5245–5256, 2024.
- [32] Z. Zhao, Y. Lai, Y. Wang, W. Jia, and H. He, "A few-shot learning based approach to IoT traffic classification," *IEEE Commun. Lett.*, vol. 26, no. 3, pp. 537–541, Mar. 2022.
- [33] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, pp. 5–32, 2001.
- [34] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 785–794.
- [35] L. Zhang, J. Na, J. Zhu, Z. Shi, C. Zou, and L. Yang, "Spatiotemporal causal convolutional network for forecasting hourly PM2.5 concentrations in Beijing, China," *Comput. Geosciences*, vol. 155, 2021, Art. no. 104869.
- [36] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 7132–7141.
- [37] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on imagenet classification," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2015, pp. 1026–1034.
- [38] H. Wang et al., "CosFace: Large margin cosine loss for deep face recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 5265–5274.
- [39] F. Wang, X. Xiang, J. Cheng, and A. L. Yuille, "NormFace: L2 hypersphere embedding for face verification," in *Proc. 25th ACM Int. Conf. Multimedia*, 2017, pp. 1041–1049.
- [40] W. Liu, Y. Wen, Z. Yu, M. Li, B. Raj, and L. Song, "SphereFace: Deep hypersphere embedding for face recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 212–220.
- [41] R. Ranjan, C. D. Castillo, and R. Chellappa, "L2-constrained softmax loss for discriminative face verification," 2017, arXiv:arXiv:1703.09507.
- [42] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive angular margin loss for deep face recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 4690–4699.

- [43] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw.*, 2017, pp. 712–717.
- [44] J. Shi, C. Yang, Q. Yang, J. Zhang, M. Cui, and G. Xiong, "SpBiSeq: An early-stage fingerprint generation method with high robustness for encrypted mobile application traffic," in *Proc. IEEE 4th Int. Conf. Comput. Commun. Eng. Technol.*, 2021, pp. 185–193.
- [45] G. Aceto, D. Ciuonzo, A. Montieri, and A. Pescapé, "Mobile encrypted traffic classification using deep learning: Experimental evaluation, lessons learned, and challenges," *IEEE Trans. Netw. Serv. Manage.*, vol. 16, no. 2, pp. 445–458, 2019.



Gongxun Miao received the MS degree from the School of Computer Science and Technology, Shandong University of Technology, Zibo, China, in 2018. He is currently working toward the PhD degree with the School of Cyberspace, Hangzhou Dianzi University, Hangzhou, China. His current research interests include computer cyber security, information system, data mining and information security.



Guohua Wu received the BS degree from the Shandong University of Technology, Jinan, China, in 1992, the MS degree from the National Institute of Metrology, Beijing, China, in 1995, and the PhD degree from Zhejiang University, Hangzhou, China, in 1998. He is currently a professor with the School of Cyberspace, Hangzhou Dianzi University, Hangzhou, Zhejiang, China. He is also an executive director with the Information Security Laboratory. His current research interests include information system, model driven architecture, data mining, and digital health.



Zhen Zhang received the graduate degree from the Zhejiang University of Computer Science, in 2005. Now he is an associate professor with the School of Cyberspace in Hangzhou Dianzi University. His current research interests include information visualization, information security, information hiding, confidential technology, etc.



Yongjie Tong received the MS degree from the School of Physics, Nankai University, Tianjin, China, in 2019. He has been engaged in the work of data mining from 2019 to now. He is currently a data-mining engineer with Zhongfu Information Company, LTD, Jinan, China. His research interests include machine learning and optimization, graph mining and cyber security.



Bing Lu received the BEng and MS degrees from the College of Computer Science and Technology, Wuhan University of Technology, China, in 2015 and 2017. He is currently data-mining engineer of Zhongfu Information Company, LTD, Jinan, China. His research interests include machine learning and its application, data mining and information security.